

torch.mm#

<https://pytorch.org/docs/stable/generated/torch.mm.html>

Description:

Performs a matrix multiplication of the matrices input and mat2. If input is a $(n \times m)(n \times m)(n \times m)$ tensor, mat2 is a $(m \times p)(m \times p)(m \times p)$ tensor, out will be a $(n \times p)(n \times p)(n \times p)$ tensor. This function does not broadcast. For broadcasting matrix products, see torch.matmul(). Supports strided and sparse 2-D tensors as inputs, autograd with respect to strided inputs.

Function Signature

```
torch.mm(input, mat2, out_dtype=None, *, out=None) → Tensor#
```

Parameters

Keyword Arguments

Parameters

Keyword

out (Tensor, optional) – the output tensor.

Code Examples

```
>>> mat1 = torch.randn(2, 3)
>>> mat2 = torch.randn(3, 3)
>>> torch.mm(mat1, mat2)
tensor([[ 0.4851,  0.5037, -0.3633],
        [-0.0760, -3.6705,  2.4784]])
```

Notes & Warnings

NOTE

Note This function does not broadcast. For broadcasting matrix products, see `torch.matmul()`.

⚠ WARNING

Warning Sparse support is a beta feature and some layout(s)/dtype/device combinations may not be supported, or may not have autograd support. If you notice missing functionality please open a feature request.

Detailed Documentation

Documentation

Performs a matrix multiplication of the matrices `input` and `mat2`.

If `input` is a $(n \times m)$ tensor, `mat2` is a $(m \times p)$ tensor, `out` will be a $(n \times p)$ tensor.

This function does not broadcast. For broadcasting matrix products, see `torch.matmul()`.

Supports strided and sparse 2-D tensors as inputs, autograd with respect to strided inputs.

This operation has support for arguments with sparse layouts. If `out` is provided its layout will be used. Otherwise, the result layout will be deduced from that of `input`.

Sparse support is a beta feature and some layout(s)/dtype/device combinations may not be supported, or may not have autograd support. If you notice missing functionality please open a feature request.

This operator supports `TensorFloat32`.

On certain ROCm devices, when using float16 inputs this module will use different precision for backward.

`input` (Tensor) – the first matrix to be matrix multiplied

`mat2` (Tensor) – the second matrix to be matrix multiplied

`out_dtype` (dtype, optional) – the dtype of the output tensor, Supported only on CUDA and for `torch.float32` given `torch.float16`/`torch.bfloat16` input dtypes

`out` (Tensor, optional) – the output tensor.